

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Факультет радіотехніки
Кафедра радіотехнічних систем

ЗВІТ

до лабораторної роботи № 4

з дисципліни «ПЗІРТ»

«ДИСКРЕТИЗАЦІЯ АНАЛОГОВИХ СИГНАЛІВ ТА АНАЛОГОВО-ЦИФРОВЕ ПЕРЕТВОРЕННЯ»

Виконане завдання: ініціалізація TMR0 та АЦП, аналого-цифрове перетворення, передача даних через UART, перетворення коду в BCD та нормування результату

Виконав:
студент групи РЕ-41
Кравченко Артем Олександрович

Київ - 2026

Хід роботи

У середовищі розробки було підготовлено проєкт для мікроконтролера PIC18F4550 відповідно до вимог лабораторної роботи № 4.

На першому етапі було проаналізовано структуру проєкту з використанням переривань та підготовлено підпрограму ініціалізації таймера TMR0 для формування частоти дискретизації.

Для TMR0 було обрано режим роботи з переповненням та визначено значення передподільника і початкового завантаження таймера.

Далі було виконано ініціалізацію модуля АЦП: налаштовано аналоговий вхід, формат результату перетворення, тактові параметри АЦП та час вибірки.

Після цього реалізовано процедуру запуску аналого-цифрового перетворення, очікування завершення перетворення та зчитування результату з регістрів ADRESH і ADRESL.

Для відображення та контролю результатів передбачено передачу цифрових даних у термінал комп'ютера через USART, а також опрацьовано варіант із використанням переривання від модуля АЦП.

Окремо було розглянуто алгоритм перетворення 16-бітного двійкового коду в двійково-десятковий формат BCD за методом послідовних зсувів і корекції тетрад.

На завершальному етапі виконано нормування цифрового коду АЦП для переходу від сирого значення $V/2^{10}$ до фізичної величини, що дозволяє отримати більш зручне числове представлення результату.

На основі методичних вказівок сформовано структуру звіту, що відповідає логіці виконання лабораторної роботи та може бути доповнена власними фрагментами коду і скріншотами моделювання.

Фрагменты программы

<pre> 1 list p=18f4550; 2 #include <p18f4550.inc> 3 4 cblock 20 5 levelH 6 levelL 7 d1 8 d2 9 d3 10 count 11 temp 12 temp16H 13 temp16L 14 R0 15 R1 16 R2 17 seg0 18 seg1 19 seg2 20 seg3 21 arg1H 22 arg1L 23 arg2H 24 arg2L 25 res0 26 res1 27 res2 28 res3 29 endc 30 31 org 000000 32 goto start 33 34 org 000008 35 goto HighInt 36 37 start 38 call System_Init 39 call USART_Init 40 call TMR0_Init </pre>	<pre> 39 call USART_Init 40 call TMR0_Init 41 call ADC_Init 42 43 ; call case3 44 ; call case4 45 ; call case5 46 47 goto Loop 48 49 Loop 50 goto Loop 51 52 System_Init 53 bsf INTCON, PEIE 54 bsf INTCON, GIE 55 56 return 57 58 USART_Init 59 movlw 0x00 60 movwf SPBRGH 61 bcf BAUDCON, BRG16 62 bcf TXSTA, BRGH 63 movlw .38 64 movwf SPBRG 65 66 bcf TXSTA, SYNC 67 bsf RCSTA, SPEN 68 69 bcf TXSTA, TX9 70 bsf TXSTA, TXEN 71 72 return 73 74 TMR0_Init 75 movlw b'10000111' 76 movwf T0CON 77 78 bsf INTCON, TMR0IE </pre>
---	--

```

76      movwi TUCON
77
78      bsf INTCON,TMR0IE
79      bsf INTCON2,TMR0IP
80
81      movlw 0xFF
82      movwf TMR0H
83      clrf TMR0L
84
85      return
86
87  ADC_Init
88      bsf TRISA,RA0
89      bsf TRISA,RA1
90
91      movlw b'00001101'
92      movwf ADCON1
93      bsf ADCON0,ADON
94      movlw b'10111110'
95      movwf ADCON2
96
97      return
98
99
100  HighInt
101      call TMR0_Interrupt
102      ;    call ADC_Interrupt
103
104      retfie
105
106  TMR0_Interrupt
107      movlw 0xFF
108      movwf TMR0H
109      clrf TMR0L
110      bcf INTCON,TMR0IF
111
112      call case1
113      ;    call case2
114
115      return
116

```

```

114
115      return
116
117
118  case1
119
120  checkTX1
121      btfss TXSTA,TRMT
122      goto checkTX1
123      movlw 'o'
124      movwf TXREG
125
126  checkTX2
127      btfss TXSTA,TRMT
128      goto checkTX2
129      movlw 'k'
130      movwf TXREG
131
132  checkTX3
133      btfss TXSTA,TRMT
134      goto checkTX3
135      movlw '!'
136      movwf TXREG
137
138  checkTX4
139      btfss TXSTA,TRMT
140      goto checkTX4
141      movlw '\n'
142      movwf TXREG
143
144  checkTX5
145      btfss TXSTA,TRMT
146      goto checkTX5
147      movlw '\r'
148      movwf TXREG
149
150      return
151
152
153  case2

```

<pre> 152 153 case2 154 call delayADC 155 156 bsf ADCON0,GO/DONE 157 158 checkADC 159 btfss ADCON0,GO/DONE 160 goto checkADC 161 162 movff ADRESH,levelH 163 movff ADRESL,levelL 164 165 checkTX6 166 btfss TXSTA,TRMT 167 goto checkTX6 168 169 movff levelL,TXREG 170 171 return 172 173 174 ADC_Interrupt 175 call delay500 176 177 movff ADRESH,levelH 178 movff ADRESL,levelL 179 180 ; clrff seg0 ;case3 181 ; clrff seg1 182 ; clrff seg2 183 ; movff levelL,seg3 ;end case3 184 185 ; movff levelH,temp16H ;case4 186 ; movff levelL,temp16L ;end case4 187 188 ; movff levelH,arg1H ;case5 189 ; movff levelL,arg1L 190 ; movlw b'00000100' 191 ; movwf arg2H </pre>	<pre> 190 ; movlw b'00000100' 191 ; movwf arg2H 192 ; movlw b'11100010' 193 ; movwf arg2L 194 ; 195 ; call mult1616 196 ; 197 ; movff res2,temp16H 198 ; movff res1,temp16L ;end case5 199 ; 200 ; 201 202 ; call bin_to_bdc ;block for case4 and case5 203 ; 204 ; movlw 0xf0 205 ; andwf R1,w 206 ; movwf seg0 207 ; swapf seg0,f 208 ; 209 ; movlw 0x0f 210 ; andwf R1,w 211 ; movwf seg1 212 ; 213 ; movlw 0xf0 214 ; andwf R2,w 215 ; movwf seg2 216 ; swapf seg2,f 217 ; 218 ; movlw 0x0f 219 ; andwf R2,w 220 ; movwf seg3 ;end block for case4 and case5 221 222 call sendUSART 223 224 ; call delay500 225 226 bcf PIR1,ADIF 227 bsf ADCON0,GO/DONE 228 229 return </pre>
---	--

```

227     bsf ADCON0,GO/DONE
228
229     return
230
231
232     case3
233         bcf INTCON,TMR0IE
234         bcf INTCON2,TMR0IP
235
236         bsf PIE1,ADIE
237         bsf IPRI,ADIP
238
239         bsf ADCON0,GO/DONE
240
241         return
242
243
244     case4
245         bcf INTCON,TMR0IE
246         bcf INTCON2,TMR0IP
247
248         bsf PIE1,ADIE
249         bsf IPRI,ADIP
250
251         bsf ADCON0,GO/DONE
252
253         return
254
255     case5
256         bcf INTCON,TMR0IE
257         bcf INTCON2,TMR0IP
258
259         bsf PIE1,ADIE
260         bsf IPRI,ADIP
261
262         bsf ADCON0,GO/DONE
263
264         return
265
266     mult1616

```

```

265
266     mult1616
267         movf arg1L,w
268         mulwf arg2L
269
270         movff PRODH,res1
271         movff PRODL,res0
272
273         movf arg1H,w
274         mulwf arg2H
275
276         movff PRODH,res3
277         movff PRODL,res2
278
279         movf arg1L,w
280         mulwf arg2H
281
282         movf PRODL,w
283         addwf res1,f
284         movf PRODH,w
285         addwfc res2,f
286         clrf WREG
287         addwfc res3,f
288
289         movf arg1H,w
290         mulwf arg2L
291
292         movf PRODL,w
293         addwf res1,f
294         movf PRODH,w
295         addwfc res2,f
296         clrf WREG
297         addwfc res3,f
298
299         return
300
301     sendUSART
302
303     checkTX13
304         btfss TXSTA,TRMT

```

```

303 checkTX13
304     btfss TXSTA,TRMT
305     goto checkTX13
306
307     movlw '\f'
308     movwf TXREG
309
310 checkTX7
311     btfss TXSTA,TRMT
312     goto checkTX7
313
314     movlw 0x30
315     addwf seg0,w
316     movwf TXREG
317
318 checkTX8
319     btfss TXSTA,TRMT
320     goto checkTX8
321
322     movlw 0x30
323     addwf seg1,w
324     movwf TXREG
325
326 checkTX9
327     btfss TXSTA,TRMT
328     goto checkTX9
329
330     movlw 0x30
331     addwf seg2,w
332     movwf TXREG
333
334 checkTX10
335     btfss TXSTA,TRMT
336     goto checkTX10
337
338     movlw 0x30
339     addwf seg3,w
340     movwf TXREG
341
342 checkTX11

```

```

341     movlw 10000
342
343 checkTX11
344     btfss TXSTA,TRMT
345     goto checkTX11
346
347     movlw '\n'
348     movwf TXREG
349
350 checkTX12
351     btfss TXSTA,TRMT
352     goto checkTX12
353
354     movlw '\r'
355     movwf TXREG
356
357     return
358
359 bin_to_bdc
360     bcf STATUS,C
361     movlw .16
362     movwf count
363     clrf R0
364     clrf R1
365     clrf R2
366
367 loop16a2
368     rlcw temp16L,f
369     rlcw temp16H,f
370     rlcw R2,f
371     rlcw R1,f
372     rlcw R0,f
373     decfsz count,f
374     goto AdjDec2
375     return
376
377 AdjDec2
378     movlw R2
379     clrf FSR0H
380     movwf FSR0L
381     call AdjBCD2
382     movlw R1
383     movwf FSR0L

```

```

378     call AdjBCD2
379     movlw R1
380     movwf FSR0L
381     call AdjBCD2
382     movlw R0
383     movwf FSR0L
384     call AdjBCD2
385     goto loop16a2
386 AdjBCD2
387     movlw 3
388     addwf INDF0,w
389     movwf temp
390     btfsc temp,3
391     movwf INDF0
392     movlw 30
393     addwf INDF0,w
394     movwf temp
395     btfsc temp,7
396     movwf INDF0
397     return
398
399
400 delayADC
401     movlw d'99'
402     movwf d1
403
404 delayADC_inner
405     decfsz d1
406     return
407     goto delayADC_inner
408
409
410 delay500
411     movlw d'18'
412     movwf d3
413 delay500_outer2
414     movlw d'255'
415     movwf d2
416 delay500_outer1
417     movlw d'255'

```

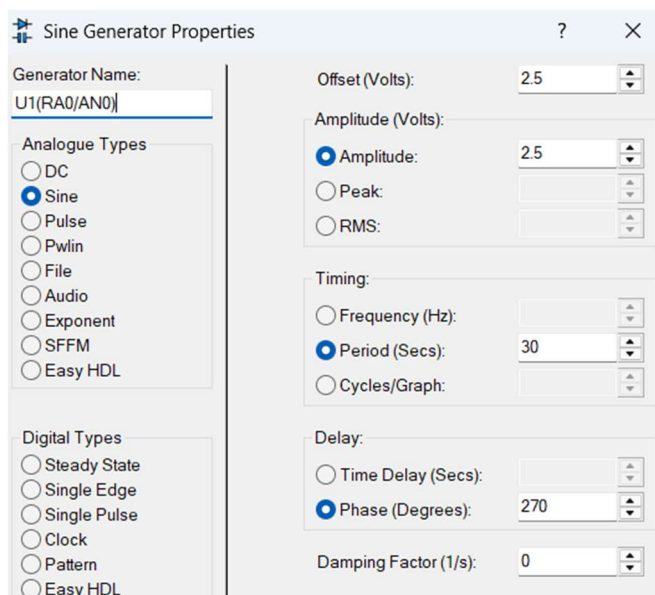
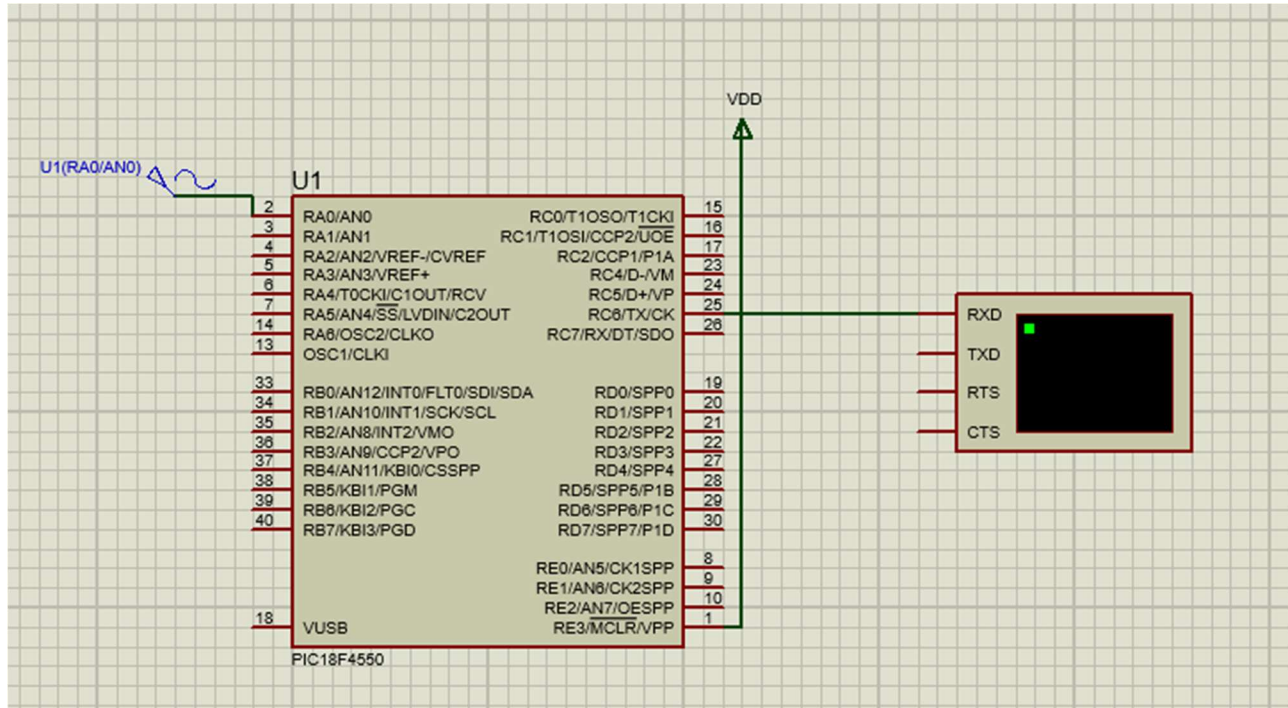
```

407     goto delayADC_inner
408
409
410 delay500
411     movlw d'18'
412     movwf d3
413 delay500_outer2
414     movlw d'255'
415     movwf d2
416 delay500_outer1
417     movlw d'255'
418     movwf d1
419 delay500_inner
420     nop
421     nop
422     decfsz d1
423     goto delay500_inner
424
425     decfsz d2
426     goto delay500_outer1
427
428     decfsz d3
429     goto delay500_outer2
430
431     return
432
433 end

```


Схема підключення та результат моделювання

У моделі використовується мікроконтролер PIC18F4550, аналоговий вхід якого підключається до джерела змінної напруги. Таймер TMR0 формує період дискретизації, модуль АЦП виконує перетворення аналогового сигналу в цифровий код, а модуль USART забезпечує передавання результатів до віртуального термінала.



Було використано саме змінне джерело напруги замість потенціометра через несправність потенціометра у версії Proteus, встановленій на моєму ПК (при запуску симуляції регулювати подільник опору неможливо).

Контрольні запитання

1. Приведіть формулу розрахунку частоти переривань таймера.

Частота переривань таймера визначається періодом його переповнення. Для TMR0 у 16-бітному режимі вона може бути подана як $f_{\text{пер}} = F_{\text{osc}} / (4 \cdot PS \cdot (65536 - N))$, де F_{osc} - тактова частота мікроконтролера, PS - коефіцієнт передподільника, N - початкове значення, завантажене в TMR0H:TMR0L.

2. Як залежить точність формування частоти дискретизації від вибору значення передподільника таймера.

Чим менший крок зміни часу переповнення можна отримати, тим точніше задається частота дискретизації. Великий передподільник збільшує діапазон затримок, але водночас зменшує точність підстроювання частоти, тому значення PS обирають як компроміс між потрібною частотою та точністю.

3. З яким міркувань обирається частота дискретизації.

Частоту дискретизації обирають з урахуванням швидкості зміни аналогового сигналу та вимог до точності його відтворення. Вона має бути достатньо великою, щоб коректно реєструвати зміну сигналу в часі, але не надмірною, щоб не перевантажувати систему зайвими перетвореннями.

4. З якою метою використовується переривання від АЦП.

Переривання від АЦП використовується для автоматичного повідомлення процесора про завершення перетворення. Це дає змогу не виконувати постійне опитування біта GO/DONE та ефективніше використовувати процесорний час.

5. Навіщо передавати дані у термінал комп'ютера через коди і виконувати перетворення даних у двійково-десятковий та ASCII коди.

Передавання даних у термінал ПК потрібне для контролю, налагодження та візуального аналізу результатів. Перетворення двійкового коду у BCD та ASCII робить числові дані зрозумілими для користувача, оскільки дозволяє відобразити їх як звичайні десяткові символи.

6. Як саме відрізняється представлення цифр у ASCII коді від їх представлення у двійково-десятковому.

У форматі BCD кожна десяткова цифра кодується окремою тетрадою, тобто 4 бітами. В ASCII кожен символ цифри має власний 8-бітний код: наприклад, символ '0' має код 30h, а символ '9' - 39h. Отже, BCD зручний для внутрішніх обчислень, а ASCII - для відображення та передавання тексту.

7. Поясніть алгоритм перетворення двійкового коду в двійково-десятковий.

Алгоритм ґрунтується на послідовному зсуві бітів вихідного числа з одночасною корекцією кожної десяткової тетради. Перед кожним новим зсувом перевіряють, чи значення тетради не перевищує 4; якщо так, до неї додають 3. Після завершення всіх зсувів отримують BCD-представлення числа.

8. Як саме зміниться алгоритм перетворення двійкового коду в двійково-десятковий перехід у разі збільшення розрядності двійкового числа.

У разі збільшення розрядності вхідного двійкового числа потрібно збільшити кількість ітерацій циклу зсуву та, за потреби, кількість BCD-тетрад для зберігання результату. Сам принцип роботи алгоритму не змінюється - змінюється лише обсяг оброблюваних даних.

9. Яка перевага використання апаратного перемножувача перед програмною реалізацією цієї процедури.

Апаратний перемножувач виконує операцію значно швидше та з меншим навантаженням на процесор, ніж програмна реалізація через цикли додавання і зсуву. Це особливо важливо в системах реального часу, де потрібно швидко обробляти результати АЦП.

Висновки

У звіті до лабораторної роботи № 4 узагальнено порядок реалізації дискретизації аналогового сигналу та аналого-цифрового перетворення на базі мікроконтролера PIC18F4550. Розглянуто налаштування таймера TMR0, ініціалізацію АЦП, передавання результатів через UART, використання переривань від АЦП, перетворення двійкового коду в BCD та нормування виміряного значення.